THE STRING CONUNDRUM

Hlomla Kuhlane
Department of Computer Science,
University of Cape Town,
Private Bag X3, Rondebosch, 7701,
South Africa
khlhlo001@myuct.ac.za

ABSTRACT

Strings are a fundamental component of programming, yet novice programmers consistently struggle to understand and use them effectively. Despite their importance, string-related difficulties remain underexplored within Computer Science Education research. This literature review examines novice programmer challenges in introductory programming, with a particular focus on string-specific difficulties and the effectiveness of existing educational tools and interventions. The reviewed studies show that string-related errors are frequent, persistent, and often resistant to traditional instruction, with evidence of poor transfer of learning across multiple contexts and programming languages. While prior research has extensively documented general programming difficulties, strings are rarely treated as a distinct area of investigation. In addition, existing educational tools primarily target general programming concepts and do not address the unique conceptual challenges associated with strings. This review identifies a significant gap between the importance of strings in introductory programming and the limited attention they receive in both research and tool design. These findings highlight the need for more focused investigation and targeted interventions to support student understanding of strings.

Keywords

Strings, novice programmers, CS1, misconceptions, programming difficulties, educational tools, Computer Science Education

1. INTRODUCTION

1.1 What is the project

Strings are a fundamental data type present in almost every programming language. They are essential for displaying information to users, processing input, creating error messages and building text-based applications. Despite their central role in programming, research has consistently shown that novice programmers experience significant difficulties understanding string concepts. This review examines the existing body of research on these difficulties, covering novice programmer difficulties in CS1 generally, string specific difficulties, and educational tools and interventions designed to address these challenges.

1.2 Why is it important

Learning to program is widely recognised as a difficult undertaking. Watson and Li [1] found that approximately one in three students worldwide fails or withdraws from introductory programming courses and this failure rate has remained stubbornly unchanged for over fifty years despite advances in tools and pedagogy. Berssanette and de Francisco [2] similarly report failure and dropout rates of approximately thirty percent in introductory programming courses globally. These figures establish that novice programmer difficulties are not a minor or isolated problem but a persistent global challenge affecting hundreds of thousands of students every year.

Within this broader challenge, strings represent a specific and underexplored source of difficulty. Strings appear in every introductory programming course and are assessed in one hundred percent of CS1 examinations yet research has shown that students consistently struggle to understand and correctly use them. Hughes et al [14] found that string related errors accounted for one hundred percent of all runtime errors in students first Java laboratory assignment. Izu and Weerasinghe [15] found that only forty nine percent of students successfully completed a string manipulation task despite having been taught the relevant concepts just one week earlier. These findings suggest that current approaches to teaching strings are not producing genuine understanding.

1.3 Scope of this Review

This review is situated within the field of Computer Science Education, specifically within research on conceptual difficulties in introductory programming. It connects to three active areas of research. The first is the study of novice programmer difficulties and misconceptions in CS1 courses. The second is the empirical investigation of string specific errors and learning challenges. The third is the design and evaluation of educational tools and interventions to support novice programmers.

1.4 Structure of this Review

This review is organised into four main sections. Section two surveys the broader literature on novice programmer difficulties in CS1 including failure rates, common misconceptions and syntax error challenges. Section three narrows the focus to string specific difficulties presenting empirical evidence of how and why strings cause particular problems for novice programmers and examining the low transfer rate from string instruction. Section four examines existing educational tools and interventions including visual tools, game-based tools, misconception targeting tools and their limitations. Section five synthesises the findings from all sections to identify the specific gaps in existing research. Section six provides a summary of key findings. Together, these sections progressively narrow from general programming difficulties to specific challenges associated with strings.

2. NOVICE PROGRAMMER DIFFICULTIES IN CS1

Introductory programming courses have long been recognised as among the most challenging courses in undergraduate education. This section surveys three interconnected aspects of these difficulties: the scale of failure rates, the nature of common

misconceptions, and the specific challenges posed by syntax and error handling.

2.1 Failure Rates and Persistence Difficulties

The high failure rates of introductory programming courses are well documented in the literature. Watson and Li [1] conducted a systematic review of 161 CS1 courses across 51 institutions in 15 countries and found a mean worldwide pass rate of 67.7%, meaning approximately one in three students fails or withdraws from introductory programming. Critically, their analysis showed that this failure rate has not significantly changed over time despite advances in tools and pedagogy, and that it is not substantially affected by the programming language taught. This last finding is particularly significant as it suggests the problem lies not in the choice of language but in deeper conceptual difficulties that students encounter regardless of which language is used. This conclusion is reinforced by

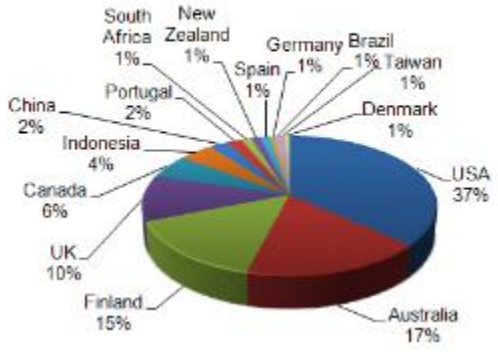


Figure 1: Geographical distribution of the outcomes, adapted from Watson and Li [1]

Berssanette and de Francisco [2], whose systematic review of active learning interventions between 2014 and 2019. Their review further highlights that high cognitive load, lack of engagement and insufficient feedback are among the key contributors to these failure rates. They find that active learning approaches improve outcomes compared to passive instruction, though failure rates remain persistently high even with these interventions.

Critically comparing these two studies, Watson and Li [1] provide a stronger and more generalisable dataset covering 161 courses across 15 countries using rigorous statistical analysis, while Berssanette and de Francisco [2] focus more narrowly on active learning interventions and cover only the period up to 2019. Watson and Li [1] are therefore the stronger source for establishing worldwide failure rate statistics, while Berssanette and de Francisco [2] are more useful for understanding what types of interventions have been attempted and why they have not fully resolved the problem.

2.2 Common Misconceptions in CS1

Beyond failure rates, a substantial body of research has investigated the specific misconceptions that novice programmers develop. Qian and Lehman [3] conducted a comprehensive literature review of student misconceptions across introductory programming and identified three categories of knowledge that students struggle with. The first is syntactic knowledge relating to the rules and symbols of the language. The second is conceptual knowledge relating to the meaning and behaviour of programming constructs. The third is strategic knowledge relating to when and how to apply programming techniques. Their review identifies string related errors explicitly, noting that mismatched quotation marks are among the most common syntactic errors and that confusion over the plus operator when used with strings versus numbers represents a common conceptual difficulty. They argue that prior knowledge, flawed understanding and the complexity of programming tasks are the primary causes of these misconceptions.

Building on this taxonomy, Cabo [4] investigated the persistence of early misconceptions in a study of 116 first year Python students over a full semester. The findings reveal that approximately 45% of students struggled with basic problem understanding throughout the entire semester and that around 20% retained early misconceptions that actively blocked their progress in later topics. s extends the framework proposed by Qian and Lehman [3] by demonstrating not only that misconceptions exist but that they do not resolve themselves naturally with time, compounding across the course and making later conceptual learning increasingly inaccessible.

The finding that misconceptions form and persist at university level is further contextualised by Mladenovic et al [5], who investigated variable misconceptions specifically in K-12 novice programmers using Logo and Python and found that misconceptions about fundamental data concepts begin forming very early in a programmer's education. Their study of 127 students showed that the choice of programming language affects the types of misconceptions students develop, with different languages producing different patterns of misunderstanding.

Critically comparing these three studies, Qian and Lehman [3] provide the broadest coverage through their literature review approach and their three category framework is the most widely cited and applicable across different contexts. However their review does not treat strings as a distinct category requiring dedicated investigation. Cabo [4] provides the strongest longitudinal evidence that misconceptions persist, but covers only one institution and focuses on variables rather than strings. Mladenovic et al [5] cover the youngest age group and use the smallest sample of 127 students, making their findings the least directly generalisable to university level CS1. Taken together the three studies establish that misconceptions are widespread, begin early and persist, but none of them treats string misconceptions as a specific research focus.

2.3 Syntax and Error Difficulties

Novice programmers face significant difficulties specifically with syntax and error interpretation. Stefik and Siebert [6] conducted four empirical studies across six programming languages to investigate whether programming language syntax affects novice accuracy. Their findings showed that Python and Ruby were significantly easier for novices than Java and Perl, and crucially that C-style syntax was no better than randomly chosen keywords for novice learners. This empirical evidence challenges the assumption that traditional syntax choices are inherently natural or learnable.

Altadmri and Brown [7] provide the largest scale empirical evidence of novice programming errors through their analysis of over 37 million compilation events from more than 250,000 students. Two of their eighteen identified error categories are

directly relevant to strings. Mistake C, mismatched brackets and quotation marks, was the single most frequent error in the entire dataset with 793,232 instances affecting 119,407 distinct students. Mistake B, using == instead of .equals to compare strings, affected 29,627 distinct students and had a median time to fix of over 1000 seconds meaning more than half of all instances were never fixed within the observation period.

Denny, Luxton-Reilly and Tempero [8] analysed 53,888 Java code submissions from 385 students and found that even the highest ability students took equally as long as lower ability students to fix the two most common errors. This result suggests that certain errors represent fundamental conceptual barriers rather than simple inexperience. Kohn [9] extended the study of error messages to Python specifically and found that missing string delimiters caused 120 name error instances, meaning students trying to fix string errors were given completely wrong advice by the Python compiler. In 25% of cases students followed that incorrect advice and made the problem worse.

Critically comparing these four studies, Altadmri and Brown [7] provide by far the strongest evidence base with 37 million compilations from 250,000 students across multiple institutions, making their findings the most generalisable. Stefik and Siebert [6] complement this by explaining why some languages are syntactically harder than others but their study focuses on overall language difficulty rather than specific error types. Denny et al [8] add the important dimension of time spent on errors but cover only one institution and use Java not Python. Kohn [9] is the only study focused specifically on Python error messages and provides unique evidence about string specific misleading feedback but uses only 100 to 150 high school students making it the smallest and least generalisable of the four. Together however they build a strong multi-institutional multi-language picture of syntax and error difficulties with consistent implications for string learning.

3. STRING SPECIFIC DIFFICULTIES

The evidence presented in Section 2 establishes that novice programmers face widespread difficulties in CS1 generally. This section narrows the focus specifically to strings, presenting empirical evidence that strings represent a particularly serious and underexplored source of difficulty, examining why strings are conceptually challenging, and demonstrating that current string instruction produces inadequate transfer of understanding.

3.1 Strings as Universal CS1 Concept

Before examining the nature of string difficulties, it is important to establish that strings are a universal and unavoidable component of CS1 education. Luxton-Reilly and Petersen [13] analysed the syntax components of 9 final examinations from 3 universities teaching introductory Python and found that strings appeared in 100% of all exams analysed. String indexing and slicing operators were assessed at all three institutions. This confirms that strings are not an optional or peripheral topic but a fundamental concept that every CS1 student must engage with. However the study also revealed that the average CS1 exam question requires students to understand 10.85 different concept components simultaneously. This means strings are never assessed in isolation but always bundled with many other concepts, making it impossible for teachers to diagnose whether a student specifically struggles with strings or with something else.

3.2 Empirical Evidence of String Difficulties

Multiple independent studies from different countries, languages and institutional contexts provide converging empirical evidence that strings cause severe and persistent difficulties for novice programmers.

Hughes, Jadud and Rodrigo [14] conducted a qualitative case study analysis of 143 first year Java students and found that string formatting errors accounted for 8% of all syntax errors and 100% of all runtime errors in the students first laboratory assignment. Through detailed analysis of individual student sessions the study shows that students spent up to 97 minutes struggling with string formatting without ever developing a correct understanding of the underlying concept. Students consistently confused string concatenation syntax with method parameter syntax, produced conditioned incorrect responses to compiler error messages, and frequently adopted avoidance strategies rather than resolving the string error.

Izu and Weerasinghe [15] examined how 338 first year engineering students approached a string manipulation task in C after one semester of instruction. Their analysis revealed that only 49% of students successfully completed the task. A particularly significant finding was that only 6% of students used string library functions despite having been explicitly taught these functions just one week earlier. The study also revealed that students held two fundamentally different approaches to strings. Fifty three percent worked at the character level treating strings as arrays of individual characters, while 32% worked at the string level treating strings as objects with associated methods. This split reveals that students do not have a uniform understanding of what a string is.

Altadmri and Brown [7] as discussed in Section 2.3 provide additional string specific evidence. Mistake B, using == instead of .equals to compare strings in Java, affected 29,627 distinct students and was one of the hardest errors to fix in the entire dataset of 37 million compilations. Kohn [9] similarly provides string specific evidence showing that missing string delimiters in Python caused 120 name error instances and that students received fundamentally misleading feedback that did not help them understand the nature of their mistake. Blok and Fehnker [23] adapted a static analysis tool to detect novice specific errors and explicitly identified using == instead of .equals to compare strings as a key error requiring better feedback. Their study confirms that all 24 student projects contained novice specific warnings while only 2 of 6 professional projects did, validating that these errors are genuinely characteristic of novice programmers rather than universal mistakes.

Mistake	Freq.	Distinct Users	Median Time to Fix	Error Type
C	793232	119407	17	Syntax
I	464075	66977	58	Type
O	342891	57129	37	Semantic
A	173938	45082	111	Syntax
N*	121663	27369	≥ 1000	Semantic
B*	121172	29627	≥ 1000	Semantic
M	86625	26527	48	Semantic
R	79462	17521	104	Semantic
P	52862	21904	24	Syntax
E*	49375	18350	401	Syntax
K	38001	19502	51	Syntax
D*	29605	8742	≥ 1000	Syntax
J	18955	10232	34	Syntax
Q	16996	7556	115	Type
L	4214	3317	12	Syntax
F	2719	2244	36	Syntax
H	1097	943	22	Syntax
G	118	112	26	Syntax

Figure 2: Statistics for mistakes committed by Black-box students between 1st Sep. 2013 to 31st Aug. 2014 (inclusive), adapted from Altadmri and Brown [7]

Critically comparing these studies, Hughes et al [14] provide the richest qualitative insight into how students experience string difficulties through detailed case studies but their sample of 143 students from a single institution in the Philippines limits generalisability. Izu and Weerasinghe [15] provide a larger sample of 338 students and the only study that directly assesses string manipulation as a standalone task, making it the most directly relevant empirical source. Altadmri and Brown [7] provide the strongest statistical evidence by far but string errors appear among eighteen other error types and are not the primary focus. Kohn [9] is the only Python specific study addressing string error feedback and fills a gap left by the Java focused studies. Blok and Fehnker [23] are published in a weaker venue than the others and cover only 24 student projects but provide useful validation that string comparison errors are novice specific. Together the five studies cover Java, Python and C across multiple countries and institutional contexts, providing convergent evidence of string difficulties that is stronger than any single study could provide.

3.3 Why Strings Are Conceptually Difficult

Sorva [10] argues through the notional machine framework that novice programmers struggle when they lack a correct dynamic model of how a concept works during program execution. Applied briefly to strings this explains why students who do not understand that a string is an indexed sequence of characters stored in memory will struggle with indexing, slicing, comparison and immutability regardless of how many times they encounter the syntax. The split between character level and string level thinking identified by Izu and Weerasinghe [15] directly reflects the kind of inconsistent and incomplete conceptual understanding that Sorva [10] identifies as a barrier to further learning.

Vainio and Sajaniemi [11] further show that novice programmers struggle to trace program execution because their working memory is easily overwhelmed when tracking multiple values simultaneously. String operations such as indexing and slicing require students to track the original string, the index position and the resulting value at the same time, which directly exceeds the cognitive capacity of many novice learners. This provides an additional explanation for why string operations are disproportionately difficult even when students appear to understand the individual components involved.

Lu and Krishnamurthi [12] extend this understanding by demonstrating that misconceptions about mutation and aliasing, which are core to understanding strings as objects in languages like Java and Python, are widespread even among advanced students. Their study found these misconceptions consistent across three different populations at different institutions, suggesting that certain conceptual difficulties in programming are deeply rooted and not specific to any one teaching context.

3.4 Low Transfer From String Instruction

A consistent finding across multiple studies is that teaching strings through traditional instruction does not result in genuine conceptual understanding. Izu and Weerasinghe [15] found that only 6% of students used string library functions in their exam despite having completed a workshop specifically focused on those functions just one week earlier. Students preferred character level approaches they had practised repeatedly throughout the course over the string level approaches they had been taught more recently. This suggests that repetition and sustained practice with a concept leads to better transfer than a single instruction episode.

Hughes et al [14] similarly demonstrate low transfer through detailed case studies of individual students. Despite receiving instruction on string formatting, students were unable to correctly construct even simple format strings in their laboratory session and reverted to trial and error strategies making repeated small changes without understanding the underlying principle. The authors conclude that string formatting involves what amounts to learning a second language embedded within Java, and that this added complexity significantly exceeds what most novice programmers can absorb through standard instruction alone.

Critically comparing these two studies, both reach the same conclusion that transfer from string instruction is poor but they do so through different methods and in different languages. Izu and Weerasinghe [15] provide quantitative evidence from a large cohort of 338 students in C while Hughes et al [14] provide rich qualitative case study evidence from 143 students in Java. The consistency of the finding across two different languages and two different methodological approaches strengthens confidence that low transfer from string instruction is a genuine and language independent problem rather than an artifact of a specific teaching context.

4. EDUCATIONAL TOOLS AND INTERVENTIONS IN PROGRAMMING EDUCATION

The evidence presented in Section 2 and 3 establishes both the severity of novice programmer difficulties generally and string difficulties specifically. This section examines the existing landscape of educational tools and interventions designed to address these difficulties, evaluates their effectiveness, and identifies their limitations.

4.1 Visual and Interactive Tools

A substantial body of research supports the effectiveness of visual and interactive tools for programming education. Scherer et al [16] conducted a large scale meta-analysis of 139 interventions and 375 effect sizes and found that visualization tools had moderate to large positive effects on programming learning with effect sizes ranging from 0.44 to 0.72. Collaboration, metacognition instruction and problem solving outside regular lessons were identified as particularly effective approaches. This large scale statistical evidence provides the strongest available justification for incorporating visual and interactive elements in educational programming tools.

Guo [17] presents Online Python Tutor, a free web based program visualization tool for Python that has been used by over 200,000 people across more than 25 universities. Students who interacted more with program visualizations scored higher on midterm exams. A particularly relevant detail is that the University of Toronto specifically customised the tool to represent strings as character arrays, confirming that visualising the internal structure of strings is considered pedagogically valuable by experienced educators. However Online Python Tutor is a general purpose visualizer and is not specifically designed to address string difficulties or string misconceptions.

Moreno, Myller, Sutinen and Ben-Ari [18] present the Jeliot 3 program visualization tool and establish a critical design principle: visualization only improves learning when students are actively engaged with it rather than passively watching it. Their iterative research driven development process demonstrated that novice students need complete and consistent visualization of both data flow and control flow and that reducing cognitive load through clear consistent visual representation is essential for novice learners.

Murali et al [19] present the Flowgramming tool which uses visual flowcharts to help novice programmers focus on problem solving logic rather than syntax. Their study found that visual tools help students separate the conceptual task of designing a solution from the syntactic task of expressing it in code, which is particularly relevant to strings where syntactic complexity often distracts students from understanding what strings actually are and how they behave.

Critically comparing these four studies, Scherer et al [16] provide the strongest and most generalisable evidence through their meta-analysis of 139 interventions but their findings cover programming education broadly rather than strings specifically. Guo [17] provides strong adoption evidence with 200,000 users but relies primarily on correlational and anecdotal data rather than controlled experimental evidence of learning outcomes. Moreno et al [18] establish important design principles through iterative empirical evaluation of Jeliot but their paper predates many modern web based tools and the evaluation evidence is limited by the absence of formal outcome measures. Murali et al [19] have the weakest evidence base being a beta test without formal evaluation but provide the most direct evidence that visual tools help with the syntax versus concept distinction that is central to string difficulties.

4.2 Game Based and Motivational Tools

Game based and gamified educational tools have been shown to significantly improve student motivation and engagement in programming courses. Agbo, Oyelere, Suhonen and Tukiainen [20] designed and evaluated iThinkSmart, a virtual reality game based application for teaching computational thinking concepts to 47 computer science students. Students using the VR game showed significantly better performance than the control group with a nearly large effect size of 0.74. The study also showed that game based learning increased student motivation, cognitive benefits, interest and attitude towards learning. The study used Design Science Research methodology which involves iterative cycles of design development and evaluation with user testing.

Hellín et al [21] present a web based gamified programming tool used with 215 undergraduate students that combined automated assessment with gamification elements including points, badges and leaderboards. Their survey found that 86.72% of students reported increased motivation to study, 75.22% reported improved self confidence from earning points, and 72.56% reported feeling motivated to find and fix errors when their code failed tests.

Critically comparing these two studies, Agbo et al [20] provide stronger experimental evidence through a controlled pre and post test design with a comparison group and a statistically significant effect size, and their paper is published in a reputable Springer journal. Hellín et al [21] use a larger sample of 215 students but their study has no control group and is published in MDPI Sustainability which is not a specialist CS education venue. The absence of a control group means it is difficult to attribute the motivational improvements in Hellín et al [21] specifically to gamification rather than to other factors such as the novelty of a new tool or the additional practice opportunity it provided. Despite this limitation the consistency of positive findings across both studies supports the general conclusion that gamification improves motivation in programming education.

4.3 Tools Addresing Misconceptions Directly

Beyond general visual and game based tools, a smaller set of tools has been specifically designed to identify and correct programming misconceptions. Lu and Krishnamurthi [12] present the SMoL Tutor, an automated interactive tutoring system built over four years of iterative study with multiple student populations. The tutor was designed explicitly to identify and correct misconceptions about core programming language behaviour. A key innovation is the concept of misinterpreters. These are formal definitions of misconceptions written as runnable code that intentionally implement incorrect semantics corresponding to a specific misconception. Using this approach the authors identified and confirmed 11 distinct misconceptions and showed that the tutor measurably corrected most of them across three different student populations. The refutation text approach used in the tutor, which directly addresses the incorrect conception rather than simply presenting the correct one, was found to be more effective than standard instruction.

Lyu et al [25] evaluated CodeTutor, an AI powered tutoring tool built on GPT-3.5, in a semester long field study with 50 undergraduate students split into control and experimental groups. Students using CodeTutor showed significant score improvements compared to the control group. The tool helped students with syntax mastery and code comprehension. However the study also found that students questioned the tool's ability to develop critical thinking and over time preferred human teaching assistants to the AI tool.

Critically comparing these two studies, Lu and Krishnamurthi [12] provide considerably stronger evidence through their four year multi-population study and their formal misinterpreter methodology gives the work a rigour and reproducibility that Lyu

et al [25] do not match. Lyu et al [25] have a small sample of only 50 students from one institution and used GPT-3.5 which may produce inaccurate responses, limiting confidence in their findings. However Lyu et al [25] are the only study to evaluate an AI powered tool in a real semester long classroom setting which gives their findings practical relevance that the more controlled SMoL Tutor studies do not fully capture. Neither tool addresses string specific misconceptions.

4.4 Limitation of Existing Tools

Despite the availability of many educational programming tools the evidence suggests that significant gaps remain. Alasmari, Singer and Bikanga Ada [22] conducted a systematic review and case study of seven online coding platforms and found that all seven platforms were missing key features identified as important for novice learners including visual maps, underlining of syntax errors and auto-completion. None of the platforms reviewed addressed string specific difficulties.

Blok and Fehnker [23] demonstrate that while static analysis tools can detect string errors they cannot explain the underlying conceptual misunderstanding to the student. Students who receive a warning about string comparison errors are not told what is conceptually wrong or how strings should be compared and why. Kohn [9] reveals that Python error messages for string errors are systematically misleading and no existing tool specifically addresses this problem. Kölling [24] articulates a set of design principles for educational programming environments including concepts before syntax, avoiding cognitive overload, quick feedback loops and consistent mental models. Despite the value of these principles Greenfoot is a general purpose Java environment and does not address string specific difficulties.

Critically comparing these four sources, Alasmari et al [22] provide the most systematic evaluation of existing tools covering seven platforms with a structured framework but their sample of seven platforms is small and the study was published in a conference rather than a journal. Blok and Fehnker [23] are published in a weaker venue and use only 24 student projects but provide unique evidence about the gap between error detection and conceptual explanation that is not covered by the other sources. Kohn [9] is the only source specifically addressing Python string error feedback limitations. Kölling [24] is published in ACM TOCE which is the strongest venue among the four but provides design principles rather than empirical evidence of tool limitations. Together the four sources establish from different angles that existing tools are insufficient for addressing string specific conceptual difficulties.

5. GAPS IN EXISTING RESEARCH

Synthesising the findings from section 2 to section 4 reveals several important gaps in the literature. The preceding sections have established a comprehensive picture of what is known about novice programmer difficulties, string specific difficulties and existing educational tools.

The first and most fundamental gap is that no study has specifically investigated string misconceptions as a primary research focus. Hughes et al [14] and Izu and Weerasinghe [15] identify that students struggle with strings and document the outcomes of that struggle, but neither study systematically investigates what specific misconceptions students hold about strings or what causes those misconceptions. Qian and Lehman's [3] comprehensive review of CS1 misconceptions mentions strings briefly but does not treat them as a distinct category requiring dedicated investigation. The result is that while the existence of string difficulties is well documented the specific nature of those difficulties remains poorly understood.

The second gap is that string difficulties are never assessed or studied in isolation. Luxton-Reilly and Petersen [13] demonstrate that the average CS1 exam question requires students to understand 10.85 different concept components simultaneously and that strings always appear in combination with other concepts. This means that no assessment instrument currently exists that can isolate string understanding from other programming concepts, making it impossible to diagnose string specific difficulties accurately.

The third gap is that string errors are disproportionately hard to fix but the reasons are not understood. Altadmri and Brown [7] show that string comparison errors took over 1000 seconds to fix on average and Denny et al [8] show that even high ability students struggle equally with certain errors. However neither study investigates why these errors are so persistent or what conceptual misunderstanding drives them.

The fourth gap is that low transfer from string instruction is documented but unexplained. Izu and Weerasinghe [15] find that only 6% of students applied string functions they had been taught one week earlier, and Hughes et al [14] show students reverting to trial and error despite having received instruction. Neither study investigates why transfer fails so dramatically for strings or what instructional approach might produce better transfer.

The fifth gap is that no educational tool exists that specifically targets string difficulties. All tools reviewed in Section 4 are general purpose programming tools. Guo's [17] Online Python Tutor visualises all Python code but is not designed around string specific challenges. Lu and Krishnamurthi's [12] SMoL Tutor addresses scope, mutation and aliasing but not strings. Alasmari et al [22] find that no existing platform addresses all novice programmer difficulties and none address string difficulties specifically.

The sixth gap is that failure rates have not improved despite fifty years of educational tool development. Watson and Li [1] demonstrate that the worldwide CS1 pass rate has remained at approximately 67.7% unchanged over time. Berssanette and de Francisco [2] confirm that even active learning interventions have not resolved this problem. This strongly suggests that the general tools and interventions developed so far are insufficient and that more targeted concept specific interventions are needed.

6. CONCLUSION

This review has examined novice programmer difficulties in CS1, string specific difficulties, and existing educational tools and interventions. The evidence across all three areas points consistently toward the same conclusion that strings are a universally important, persistently difficult and critically underexplored concept in introductory programming education.

Novice programmer difficulties in CS1 are severe and widespread, with Watson and Li [1] documenting failure rates of approximately one in three students across 15 countries, and Cabo [4] showing that misconceptions form early and persist throughout a course if left unaddressed. String related errors are among the most frequent and hardest to fix in large scale datasets as shown

by Altadmri and Brown [7], and string instruction consistently produces poor transfer of understanding as independently demonstrated by Hughes et al [14] and Izu and Weerasinghe [15]. Despite this, no study has investigated string misconceptions as a primary focus, string difficulties have never been studied in isolation, and no educational tool exists that specifically targets string understanding.

Existing educational tools have demonstrated positive effects on programming learning broadly as shown by Scherer et al [16] and Lu and Krishnamurthi [12], but a critical examination reveals that none were designed with string difficulties in mind. The gap between what tools can currently do and what string learners specifically need remains entirely unfilled.

Strings are too important and the difficulties too consistent and too severe to remain as neglected as the current literature suggests they are. The six gaps identified in this review collectively represent a significant and timely opportunity for dedicated research and intervention within Computer Science Education.

These findings highlight the need for focused research into string-specific misconceptions and the development of targeted educational interventions.

7. REFERENCES

[1] Christopher Watson and Frederick W.B. Li. 2014. Failure Rates in Introductory Programming Revisited. In *Proceedings of the 2014 Conference on Innovation and Technology in Computer Science Education*. ACM, New York, NY, USA, 39–44.

[2] João Henrique Berssanette and Antonio Carlos de Francisco. 2021. Active Learning in the Teaching-Learning Process of Computer Programming: A Systematic Literature Review. *Journal of Information Technology Education: Research* 20, 1–57.

[3] Yizhou Qian and James Lehman. 2017. Students' Misconceptions and Other Difficulties in Introductory Programming: A Literature Review. *ACM Transactions on Computing Education* 18, 1, Article 1, 1–24.

[4] C. Cabo. 2026. Persistence of Early Misconceptions Hinders Progress in Learning Computer Programming. *IEEE Transactions on Education* 69, 1, 31–39.

[5] Mirela Mladenovic, Marija Baucal, and Divna Krpan. 2019. Influence of Variable Misconceptions on Mastering Other Programming Concepts. In *Proceedings of the 2019 Koli Calling International Conference on Computing Education Research*. Springer, 1–10.

[6] Andreas Stefik and Susanna Siebert. 2013. An Empirical Investigation into Programming Language Syntax. *ACM Transactions on Computing Education* 13, 4, Article 19, 1–40.

[7] Amjad Altadmri and Neil C.C. Brown. 2015. 37 Million Compilations: Investigating Novice Programming Mistakes in Large-Scale Student Data. In *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*. ACM, New York, NY, USA, 522–527.

[8] Paul Denny, Andrew Luxton-Reilly, and Ewan Tempero. 2012. All Syntax Errors Are Not Equal. In *Proceedings of the 17th ACM Annual Conference on Innovation and Technology in Computer Science Education*. ACM, New York, NY, USA, 75–80.

[9] Tobias Kohn. 2019. The Error Behind the Message: Finding the Cause of Error Messages in Python. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*. ACM, New York, NY, USA, 524–530.

[10] Juha Sorva. 2013. Notional Machines and Introductory Programming Education. *ACM Transactions on Computing Education* 13, 2, Article 8, 1–31.

[11] Vesa Vainio and Jorma Sajaniemi. 2007. Factors in Novice Programmers' Poor Tracing Skills. In *Proceedings of the 12th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education*. ACM, New York, NY, USA, 236–240.

[12] Kuang-Chen Lu and Shriram Krishnamurthi. 2024. Identifying and Correcting Programming Language Behavior Misconceptions. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1, Article 106, 1–28.

[13] Andrew Luxton-Reilly and Andrew Petersen. 2017. The Compound Nature of Novice Programming Assessments. In *Proceedings of the Nineteenth Australasian Computing Education Conference*. ACM, New York, NY, USA, 26–35.

[14] Michael C. Hughes, Matthew C. Jadud, and Ma. Mercedes T. Rodrigo. 2010. String Formatting Considered Harmful for Novice Programmers. *Computer Science Education* 20, 3, 201–228.

[15] Cruz Izu and Amali Weerasinghe. 2020. Assessing CS1 Design Skills with a String Manipulation Task. In *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education*. ACM, New York, NY, USA, 432–438.

[16] Ronny Scherer, Fazilat Siddiq, and Bárbara Sánchez Viveros. 2020. The Effectiveness of Instructional Approaches and Conditions for Learning Computer Programming. *Computers in Human Behavior* 109, 106349.

[17] Philip J. Guo. 2013. Online Python Tutor: Embeddable Web-Based Program Visualization for CS Education. In *Proceedings of the 44th ACM Technical Symposium on Computer Science Education*. ACM, New York, NY, USA, 579–584.

[18] Andrés Moreno, Niko Myller, Erkki Sutinen, and Mordechai Ben-Ari. 2004. Visualizing Programs with Jeliot 3. In *Proceedings of the Working Conference on Advanced Visual Interfaces*. ACM, New York, NY, USA, 373–376.

[19] Ritwik Murali, Rajkumar Sukumar, Mary Sanjana Gali, and Veeramanohar Avudaiappan. 2023. Empowering Novice Programmers with Visual Problem Solving Tools. In *Proceedings of the 16th Annual ACM India Compute Conference*. ACM, New York, NY, USA, 1–4.

[20] Friday Joseph Agbo, Solomon Sunday Oyelere, Jarkko Suhonen, and Markku Tukiainen. 2022. Design, Development, and Evaluation of a Virtual Reality Game-Based Application to Support Computational Thinking. *Education and Information Technologies* 27, 503–533.

[21] Carlos J. Hellín, Francisco Calles-Esteban, Adrián Valledor, Josefa Gómez, Salvador Otón-Tortosa, and Abdelhamid Tayebi.

2023. Enhancing Student Motivation and Engagement through a Gamified Learning Environment. *Sustainability* 15, 14119.

[22] Ohud Alasmari, Jeremy Singer, and Mireilla Bikanga Ada. 2023. Do Current Online Coding Tutorial Systems Address Novice Programmer Difficulties? In *Proceedings of the 15th International Conference on Education Technology and Computers*. ACM, New York, NY, USA, 1–7.

[23] Tim Blok and Ansgar Fehnker. 2017. Automated Program Analysis for Novice Programmers. In *Proceedings of the 3rd International Conference on Higher Education Advances*. HEAD17, 1–8.

[24] Michael Kölling. 2010. The Greenfoot Programming Environment. *ACM Transactions on Computing Education* 10, 4, Article 14, 1–21.

[25] Yunsung Lyu, Zhengdong Wang, Yubin Chung, Meng Sun, and Tianhao Zhang. 2024. Evaluating the Effectiveness of LLMs in Introductory Computer Science Education: A Semester-Long Field Study. In *Proceedings of the 11th ACM Conference on Learning at Scale*. ACM, New York, NY, USA